

Setup in VS Code

Hello and welcome 😊 U are about to learn a truncation of 3 different softwares! If u ever want to learn quickly and especially HOW, check out my 'vanessagetsanA' channel. And yes, I did get that A in discrete structures. 😊

```
mkdir C:\Users\vanes\analog-measurement-practice
```

```
cd C:\Users\vanes\analog-measurement-practice
```

```
mkdir simulation
```

```
mkdir processing
```

```
mkdir analysis
```

```
mkdir interface
```

```
mkdir measurements
```

```
mkdir measurements\raw
```

```
mkdir measurements\processed
```

```
mkdir results
```

```
mkdir docs
```

```
code .
```

```
Create: simulation\test_circuit.cir
```

```
* Analog Measurement Practice
```

```
* RC measurement circuit
```

```
Vinput source 0 SIN(0 0.01 1000)
```

```
Rfilter source sense 1k
```

```
Cfilter sense 0 100n
```

```
.tran 1u 5m
```

```
.print tran V(source) V(sense)
```

```
.end
```

Open terminal (vs code one):

```
& "C:\Users\vanes\Downloads\ngspice-47_64\Spice64\bin\ngspice.exe" -b
```

```
simulation\test_circuit.cir -o simulation\simulation_output.txt
```

```
Copy-Item simulation\simulation_output.txt measurements\raw\raw_measurements.txt
```

```
Create: processing\convert_data.py
```

```
from pathlib import Path
```

```
input_file = Path("measurements/raw/raw_measurements.txt")
```

```
output_file = Path("measurements/processed/voltage_data.csv")
```

```
rows = []

with input_file.open("r") as file:
    for line in file:
        parts = line.split()

        if len(parts) == 4 and parts[0].isdigit():
            sample_number, time, input_voltage, output_voltage = parts

            rows.append(
                f"{time},{input_voltage},{output_voltage}\n"
            )

with output_file.open("w") as file:
    file.write("time,input_voltage,output_voltage\n")
    file.writelines(rows)

print(f"Processed {len(rows)} measurement samples.")
print(f"Saved to: {output_file}")
```

ngspice + Cuda + python

- 1) click  and type `x64 Native Tools Command Prompt for VS`
- 2) where cl
- 3) `cd /d C:\Users\vanes\analog-measurement-practice`
- 4) `notepad simulation\simulation\test_circuit.cir`
- 5) `"C:\Users\vanes\Downloads\ngspice-47_64\Spice64\bin\ngspice.exe" -b simulation\simulation\test_circuit.cir -o simulation\simulation_output.txt`
- 6) type `simulation\simulation_output.txt`
- 7) copy `simulation\simulation_output.txt` `measurements\raw\raw_measurements.txt`
- 8) `notepad processing\convert_data.py`

`from pathlib import Path`

`input_file = Path("measurements/raw/raw_measurements.txt")`

`output_file = Path("measurements/processed/voltage_data.csv")`

`rows = []`

`with input_file.open("r") as file:`

`for line in file:`

`parts = line.split()`

`if len(parts) == 4 and parts[0].isdigit():`

`sample_number, time, input_voltage, output_voltage = parts`

`rows.append(`

`f'{time},{input_voltage},{output_voltage}\n'`

`)`

`with output_file.open("w") as file:`

`file.write("time,input_voltage,output_voltage\n")`

`file.writelines(rows)`

`print(f'Processed {len(rows)} measurement samples.')`

`print(f'Saved to: {output_file}')`

9) `python processing\convert_data.py`

10) `notepad analysis\test.cu`

`#include <stdio.h>`

`int main() {`

`printf("CUDA program completed.\n");`

`return 0;`

`}`

11) `nvcc analysis\test.cu -o analysis\test.exe`

12) analysis\test.exe

13) notepad analysis\signal_metrics.cu

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    const char* filename = "measurements/processed/voltage_data.csv";
```

```
    FILE* file = fopen(filename, "r");
```

```
    if (file == NULL) {
```

```
        printf("Error: Could not open voltage_data.csv\n");
```

```
        return 1;
```

```
    }
```

```
    char line[256];
```

```
    // Skip CSV header
```

```
    fgets(line, sizeof(line), file);
```

```
    int count = 0;
```

```
    double sum_input = 0.0;
```

```
    double sum_output = 0.0;
```

```
    double min_input = 1e9;
```

```
    double max_input = -1e9;
```

```
    double min_output = 1e9;
```

```
    double max_output = -1e9;
```

```
    double sum_input_squared = 0.0;
```

```
    double sum_output_squared = 0.0;
```

```
    double time;
```

```
    double input_voltage;
```

```
    double output_voltage;
```

```
    while (fgets(line, sizeof(line), file) != NULL) {
```

```
        if (sscanf(line, "%lf,%lf,%lf",
```

```
            &time,
```

```

        &input_voltage,
        &output_voltage) == 3) {

    sum_input += input_voltage;
    sum_output += output_voltage;

    if (input_voltage < min_input)
        min_input = input_voltage;

    if (input_voltage > max_input)
        max_input = input_voltage;

    if (output_voltage < min_output)
        min_output = output_voltage;

    if (output_voltage > max_output)
        max_output = output_voltage;

    sum_input_squared += input_voltage * input_voltage;
    sum_output_squared += output_voltage * output_voltage;

    count++;
}
}

fclose(file);

if (count == 0) {
    printf("Error: No measurement samples found.\n");
    return 1;
}

double mean_input = sum_input / count;
double mean_output = sum_output / count;

double rms_input = sqrt(sum_input_squared / count);
double rms_output = sqrt(sum_output_squared / count);

double rms_gain = rms_output / rms_input;

double peak_to_peak_input = max_input - min_input;
double peak_to_peak_output = max_output - min_output;

double peak_input = fmax(fabs(min_input), fabs(max_input));

```

```
double peak_output = fmax(fabs(min_output), fabs(max_output));
```

```
printf("CUDA signal analysis starting...\n");  
printf("Loaded %d measurement samples.\n", count);
```

```
printf("Mean Input = %.8f V\n", mean_input);  
printf("Mean Output = %.8f V\n", mean_output);
```

```
printf("RMS Input = %.8f V\n", rms_input);  
printf("RMS Output = %.8f V\n", rms_output);
```

```
printf("RMS Gain = %.6f\n", rms_gain);  
printf("RMS Gain = %.2f dB\n",  
       20.0 * log10(rms_gain));
```

```
printf("Input Peak-to-Peak = %.8f V\n",  
       peak_to_peak_input);
```

```
printf("Output Peak-to-Peak = %.8f V\n",  
       peak_to_peak_output);
```

```
printf("Input Peak Voltage = %.8f V\n",  
       peak_input);
```

```
printf("Output Peak Voltage = %.8f V\n",  
       peak_output);
```

```
FILE* results = fopen("results/analysis_results.txt", "w");
```

```
if (results == NULL) {  
    printf("Error: Could not create analysis results file.\n");  
    return 1;  
}
```

```
fprintf(results, "Analog Measurement Analysis\n");  
fprintf(results, "=====\n");  
fprintf(results, "Samples: %d\n\n", count);
```

```
fprintf(results, "Mean Input: %.8f V\n", mean_input);  
fprintf(results, "Mean Output: %.8f V\n\n", mean_output);
```

```
fprintf(results, "RMS Input: %.8f V\n", rms_input);  
fprintf(results, "RMS Output: %.8f V\n\n", rms_output);
```

```

fprintf(results, "RMS Gain: %.6f\n", rms_gain);
fprintf(results, "RMS Gain: %.2f dB\n\n",
    20.0 * log10(rms_gain));

fprintf(results, "Input Peak-to-Peak: %.8f V\n",
    peak_to_peak_input);

fprintf(results, "Output Peak-to-Peak: %.8f V\n\n",
    peak_to_peak_output);

fprintf(results, "Input Peak: %.8f V\n", peak_input);
fprintf(results, "Output Peak: %.8f V\n", peak_output);

fclose(results);

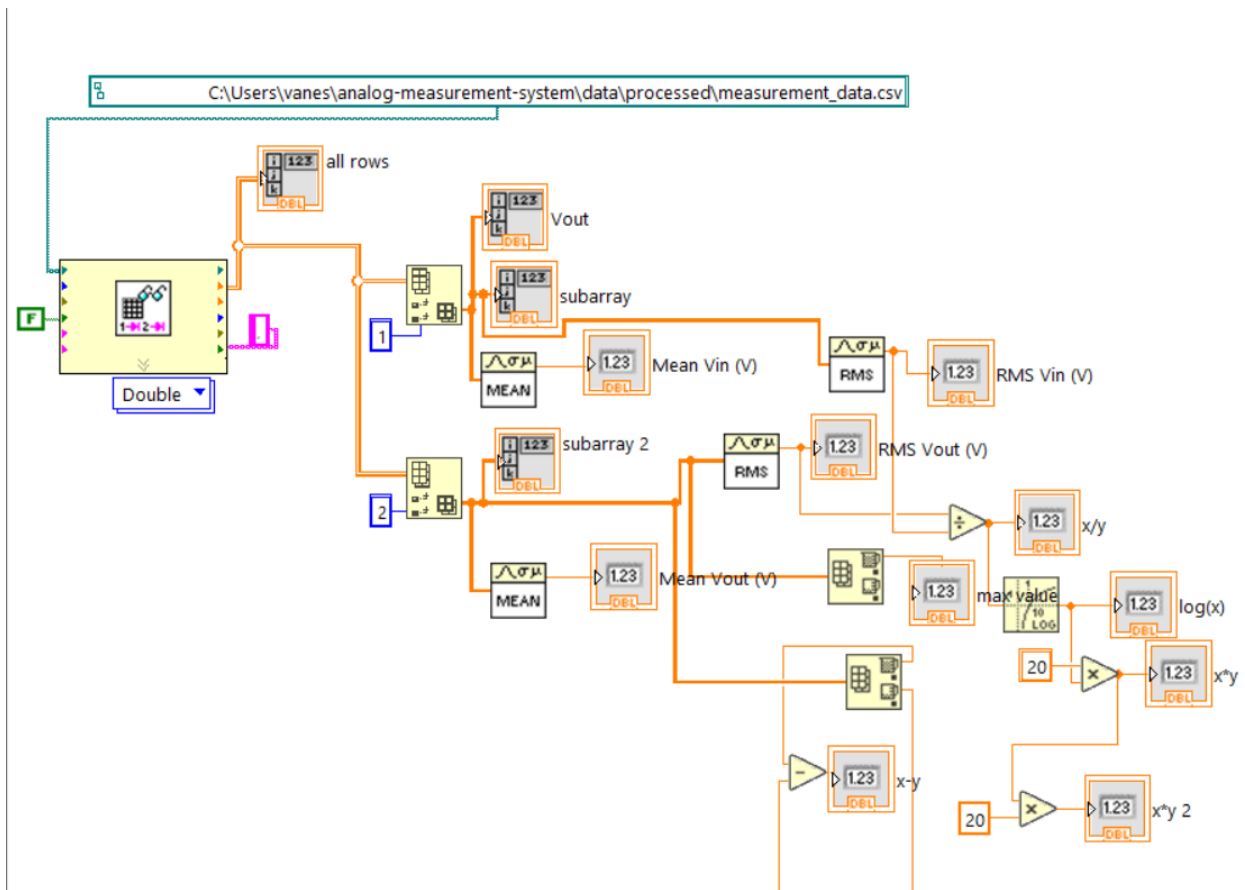
printf("Results saved to results\\analysis_results.txt\n");

return 0;
}
14) nvcc analysis\\signal_metrics.cu -o analysis\\signal_metrics.exe
15) analysis\\signal_metrics.exe
16) type results\\analysis_results.txt

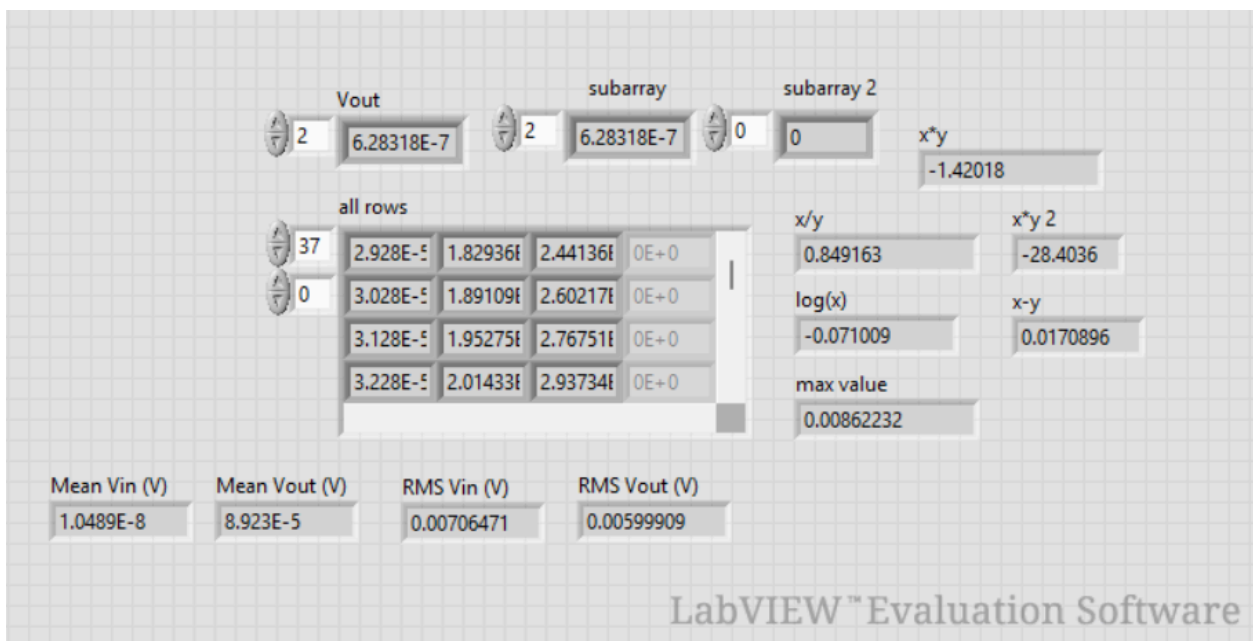
```

LabVIEW

Lastly, LabVIEW, you just want to check ur Cuda answers. And yes mine match 😎



LabVIEW™ Evaluation Software



LabVIEW™ Evaluation Software

@VIgnore - Full Reference for later

** Visual Studio 2026 Developer Command Prompt v18.10.0

** Copyright (c) 2026 Microsoft Corporation

[vcvarsall.bat] Environment initialized for: 'x64'

C:\Program Files\Microsoft Visual Studio\18\Community>where cl

C:\Program Files\Microsoft Visual

Studio\18\Community\VC\Tools\MSVC\14.51.36231\bin\Hostx64\x64\cl.exe

C:\Program Files\Microsoft Visual Studio\18\Community>cd /d

C:\Users\vanes\analog-measurement-system

C:\Users\vanes\analog-measurement-system>nvcc cuda\test.cu -o cuda\test.exe
test.cu

LINK : fatal error LNK1561: entry point must be defined

C:\Users\vanes\analog-measurement-system>notepad cuda\test.cu

C:\Users\vanes\analog-measurement-system>nvcc cuda\test.cu -o cuda\test.exe
test.cu

C:\Users\vanes\analog-measurement-system>cuda\test.exe
CUDA program completed.

C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe

signal_analysis.cu

C:\Users\vanes\analog-measurement-system\cuda\signal_analysis.cu(9): warning #550-D:
variable "signal" was set but never used

float signal[N];

^

Remark: The warnings can be suppressed with "-diag-suppress <warning-number>"

C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe

CUDA signal analysis starting...

Generated 5008 signal samples.

C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe

signal_analysis.cu

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
CUDA signal analysis starting...
Loaded 5008 measurement samples.
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
CUDA signal analysis starting...
Loaded 5008 measurement samples.
Mean Vin = 0.00000001 V
Mean Vout = 0.00008925 V
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
CUDA signal analysis starting...
Loaded 5008 measurement samples.
Mean Vin = 0.00000001 V
Mean Vout = 0.00008925 V
RMS Vin = 0.00706542 V
RMS Vout = 0.00599969 V
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
CUDA signal analysis starting...
Loaded 5008 measurement samples.
Mean Vin = 0.00000001 V
Mean Vout = 0.00008925 V
RMS Vin = 0.00706542 V
RMS Vout = 0.00599969 V
RMS Gain = 0.849163
RMS Gain = -1.42 dB
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
cuda\signal_analysis.exe
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
```

```
CUDA signal analysis starting...
```

```
Loaded 5008 measurement samples.
```

```
Mean Vin = 0.00000001 V
```

```
Mean Vout = 0.00008925 V
```

```
RMS Vin = 0.00706542 V
```

```
RMS Vout = 0.00599969 V
```

```
RMS Gain = 0.849163
```

```
RMS Gain = -1.42 dB
```

```
Vin Peak-to-Peak = 0.01999997 V
```

```
Vout Peak-to-Peak = 0.01708964 V
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
```

```
cuda\signal_analysis.exe
```

```
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
```

```
CUDA signal analysis starting...
```

```
Loaded 5008 measurement samples.
```

```
Mean Vin = 0.00000001 V
```

```
Mean Vout = 0.00008925 V
```

```
RMS Vin = 0.00706542 V
```

```
RMS Vout = 0.00599969 V
```

```
RMS Gain = 0.849163
```

```
RMS Gain = -1.42 dB
```

```
Vin Peak-to-Peak = 0.01999997 V
```

```
Vout Peak-to-Peak = 0.01708964 V
```

```
Vin Peak Voltage = 0.00999998 V
```

```
Vout Peak Voltage = 0.00862232 V
```

```
C:\Users\vanes\analog-measurement-system>nvcc cuda\signal_analysis.cu -o
```

```
cuda\signal_analysis.exe
```

```
signal_analysis.cu
```

```
C:\Users\vanes\analog-measurement-system>cuda\signal_analysis.exe
```

```
CUDA signal analysis starting...
```

```
Loaded 5008 measurement samples.
```

```
Mean Vin = 0.00000001 V
```

```
Mean Vout = 0.00008925 V
```

```
RMS Vin = 0.00706542 V
```

```
RMS Vout = 0.00599969 V
```

```
RMS Gain = 0.849163
```

```
RMS Gain = -1.42 dB
```

```
Vin Peak-to-Peak = 0.01999997 V
```

Vout Peak-to-Peak = 0.01708964 V
Vin Peak Voltage = 0.00999998 V
Vout Peak Voltage = 0.00862232 V
Results saved to results/cuda_analysis.txt

C:\Users\vanes\analog-measurement-system>

C:\Users\vanes\analog-measurement-system>type results\cuda_analysis.txt
CUDA Analog Measurement Analysis

=====

Samples: 5008

Mean Vin: 0.00000001 V
Mean Vout: 0.00008925 V

RMS Vin: 0.00706542 V
RMS Vout: 0.00599969 V

RMS Gain: 0.849163
RMS Gain: -1.42 dB

Vin Peak-to-Peak: 0.01999997 V
Vout Peak-to-Peak: 0.01708964 V

Vin Peak: 0.00999998 V
Vout Peak: 0.00862232 V

C:\Users\vanes\analog-measurement-system>^A

C:\Users\vanes\analog-measurement-system>